FIAP · ENTREGA DEVOPS · DOCUMENTAÇÃO TÉCNICA

Projeto — Cidades ESG Inteligentes

Hub de Inovação Águia Branca: pipeline CI/CD, containerização e deploy em staging e produção



A API Spring Boot que leva ideias da operação até virarem projetos acompanhados pela liderança, agora com o ciclo de vida inteiro automatizado: build, testes, imagem Docker, deploy em staging, aprovação e deploy em produção, a cada push na `main`.

INTEGRANTES

Alex Caje Felix

Carolina

Guilherme Machado

PROJETO

Repositório	github.com/AlexCajeFelix/FIAP-AGUIA-BRANCA
Stack	Java 21 · Spring Boot 3.3 · MongoDB 7
DevOps	GitHub Actions · Docker · GHCR
Data	Outubro de 2026

0 Sumário e visão geral

1	Projeto e integrantes	visão geral, arquitetura
2	Pipeline CI/CD	ferramenta, etapas e lógica
3	Docker	arquitetura, imagem criada, comandos usados
4	Prints do pipeline rodando	build, testes, imagem e deploy
5	Prints de staging e produção funcionando	evidências dos dois ambientes
6	Desafios encontrados e soluções	
7	Checklist de entrega	

1. Projeto e integrantes

Projeto: Cidades ESG Inteligentes — **Hub de Inovação Águia Branca**. **Integrantes:** Alex Caje Felix, Carolina e Guilherme Machado. **Repositório:** github.com/AlexCajeFelix/FIAP-AGUIA-BRANCA.

A aplicação escolhida é o projeto Java Spring desenvolvido anteriormente: uma API que recebe ideias da operação, leva cada uma pela decisão do gestor, promove as aprovadas a projeto e acompanha orçamento e progresso até o painel da liderança. Um app Android (Kotlin + Jetpack Compose) consome a API.

185	4	364 MB	25
testes automatizados no pipeline	jobs: build, imagem, staging, produção	imagem final (multi-stage, JRE)	verificações de smoke test pós-deploy

O que foi adaptado para simular produção

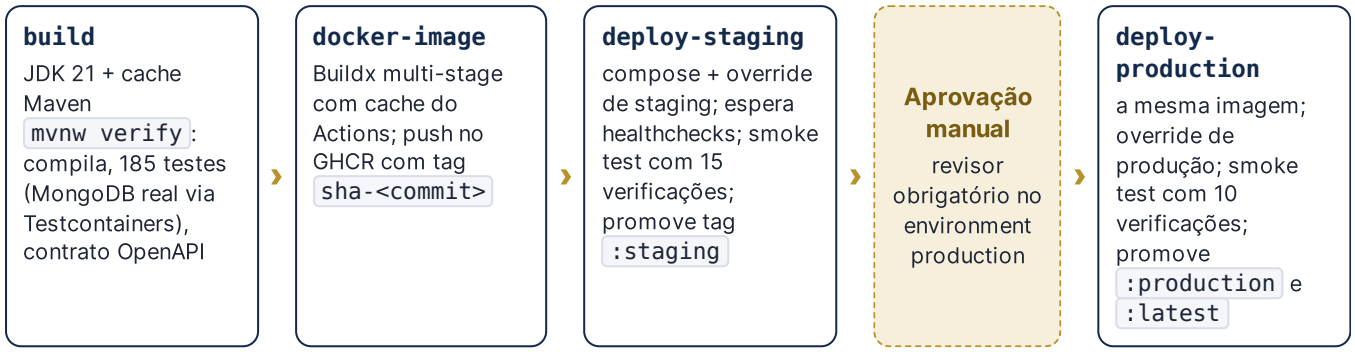
Antes	Depois desta entrega
CI só com build e testes	Pipeline CI/CD completo: build → testes → imagem no registry → deploy em staging → smoke test → aprovação → deploy em produção
Dockerfile de dois estágios simples	Jar em camadas, cache de dependências, healthcheck, JVM ciente do limite de memória, labels OCI com versão e commit
Um compose para desenvolvimento	Compose base + overrides de staging e produção, com redes separadas (banco em rede interna), volumes e segredos obrigatórios
Perfis só `dev`	Profiles `staging` (massa de demonstração para QA) e `prod` (sem nenhuma conta conhecida)
Versão fixa no contrato	Versão implantada e ambiente expostos no OpenAPI e conferidos pelo smoke test a cada deploy

Arquitetura da aplicação

App Android → API Spring Boot 3.3.5 (Java 21, JWT, RFC 7807) → MongoDB 7 em replica set de um nó (necessário para as transações de métricas). Organização por fatia vertical: `auth`, `idea`, `project`, `strategy`. A IA (Gemini) é opcional e chamada só pelo servidor.

2 Pipeline CI/CD

Ferramenta: **GitHub Actions**, num único workflow (`.github/workflows/ci-cd.yml`). Registry: **GitHub Container Registry**. Ambientes: **GitHub Environments** `staging` e `production`.



Em pull request rodam só `build` e `docker-image` (a imagem é construída, mas não publicada). Push na `main` roda o fluxo inteiro.

Gatilhos

Evento	O que executa
Pull request	`build` + `docker-image` sem push. O check `build` é obrigatório para o merge na `main`.
Push na `main`	Fluxo completo até produção.
`workflow_dispatch`	O mesmo da `main`, disparado manualmente em *Actions → CI/CD → Run workflow*.

Etapas em detalhe

Job	O que faz	Artefatos / evidência
`build`	Checkout, JDK 21 Temurin com cache do Maven, `./mvnw -B verify`: compilação com `-Xlint:all`, testes unitários, de integração (MockMvc + MongoDB via Testcontainers), matriz de autorização e geração do contrato OpenAPI.	jar, `openapi.json`, relatórios Surefire e tabela de testes no resumo do run
`docker-image`	Calcula a tag (`sha-` + 7 caracteres do commit; nome em minúsculas, exigência do GHCR), faz login com o `GITHUB_TOKEN` e builda com `docker/build-push-action`, passando versão e commit como build args.	imagem no GHCR e digest no resumo
`deploy-staging`	`scripts/deploy.sh staging <imagem>`: `docker compose up --wait` com o override de staging. Depois `scripts/smoke-test.sh` percorre login → ideia → aprovação → projeto → métricas (transação) → painel. Passando, promove a imagem para `:staging`.	print do Swagger tirado no runner, health, logs dos containers (`evidencias-staging`)
`deploy-production`	Fica em espera até a aprovação no environment `production` (que também só aceita a `main`). Mesmo deploy com o override de produção; o smoke test prova que as contas de demonstração **não** existem. Promove para `:production` e `:latest`.	`evidencias-production`

Lógica e decisões

Build once, deploy many

A imagem é construída uma vez por commit. Staging e produção rodam o **mesmo digest**; promover é apenas adicionar uma tag (`docker buildx imagetools create`), sem rebuild — o que foi testado é exatamente o que vai para produção.

Portões de qualidade

Teste vermelho barra a imagem; healthcheck que não fica verde barra o deploy (`--wait`); smoke test que falha barra a promoção; produção ainda exige aprovação humana.

Versão conferida

A imagem leva a versão (`APP_VERSION`) e a API a expõe em `info.version` e `x-environment` do OpenAPI. O smoke test compara com a tag recém-construída: se o deploy subiu a imagem errada ou no ambiente errado, falha.

Segredos fora do código

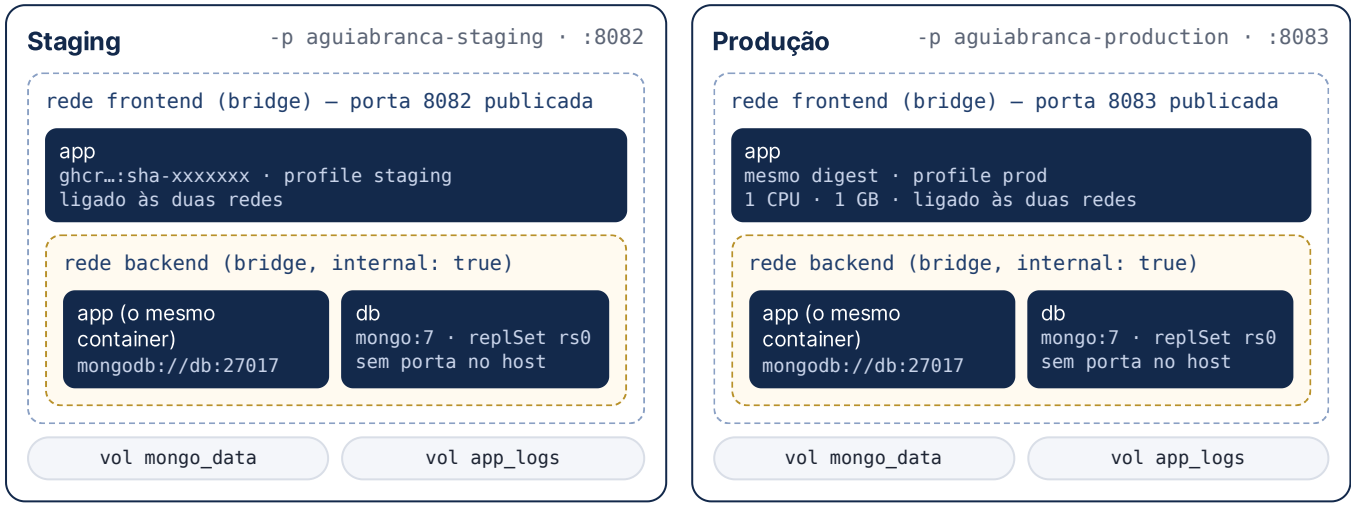
`JWT_SECRET` diferente por ambiente, guardado como secret de cada GitHub Environment. Os overrides não têm valor default: sem o secret, o deploy para antes de subir qualquer container.

Onde os ambientes rodam. Sem orçamento de nuvem, os ambientes de staging e produção são criados no runner do GitHub Actions, que funciona como host Docker descartável: cada deploy sobe o ambiente, valida com o smoke test, registra as evidências (print, health, logs) e o derruba ao fim do job. O que persiste é a imagem promovida no GHCR (`:staging`, `:production`) — o mesmo `scripts/deploy.sh` sobe essa imagem em qualquer servidor com Docker, e foi assim que os dois ambientes também foram reproduzidos lado a lado na máquina local (seção 5).

3 Docker: arquitetura, imagem e comandos

Arquitetura dos ambientes

Um único `docker-compose.yml` é a base; cada ambiente aplica um override e um nome de projeto próprio (`-p`), o que isola containers, redes e volumes. Abaixo, staging e produção como sobem no pipeline e na máquina local:



	Local (`docker-compose.yml`)	Staging (+ `.staging.yml`)	Produção (+ `.prod.yml`)
Imagem	build local	imagem do GHCR	mesma imagem de staging
Profile Spring	`dev`	`staging`	`prod`
Porta da API	8080	8082	8083
Massa de dados	seed de dev	seed de demonstração	nenhuma
Mongo exposto no host	sim (27017)	não	não
Rede `backend` interna	não	sim	sim
`JWT_SECRET`	default de dev	secret obrigatório	secret obrigatório
Restart / limites	`unless-stopped`	768 MB	`always`, 1 CPU, 1 GB, rotação de log

Dockerfile e estratégias

```
# ---------- Estágio 1: build ----------
FROM eclipse-temurin:21-jdk-alpine AS builder
COPY .mvn/ .mvn/
COPY mvnw pom.xml ./
RUN --mount=type=cache,target=/root/.m2 ./mvnw -B -q dependency:go-offline
COPY src ./src
RUN --mount=type=cache,target=/root/.m2 ./mvnw -B -q clean package -DskipTests \
    && cp target/*.jar app.jar && java -Djarmode=layertools -jar app.jar extract --destination
extracted

# ---------- Estágio 2: runtime ----------
FROM eclipse-temurin:21-jre-alpine AS runner
RUN addgroup -S appgroup && adduser -S appuser -G appgroup && mkdir -p /app/logs && chown
appuser:appgroup /app/logs
COPY --from=builder --chown=appuser:appgroup /app/extracted/dependencies/ ./        # 40 MB, quase
nunca muda
COPY --from=builder --chown=appuser:appgroup /app/extracted/spring-boot-loader/ ./
COPY --from=builder --chown=appuser:appgroup /app/extracted/snapshot-dependencies/ ./
COPY --from=builder --chown=appuser:appgroup /app/extracted/application/ ./         # < 1 MB, muda a
cada commit
USER appuser
ENV SERVER_PORT=8080 JAVA_OPTS="-XX:MaxRAMPercentage=75 -XX:+ExitOnOutOfMemoryError"
HEALTHCHECK --interval=10s --start-period=60s CMD wget -qO- http://127.0.0.1:8080/actuator/health |
grep -q UP
ENTRYPOINT ["sh", "-c", "exec java $JAVA_OPTS org.springframework.boot.loader.launch.JarLauncher"]
```

Estratégia	Motivo
Multi-stage (JDK só no build)	Imagem final de 364 MB, contra 553 MB só da base com JDK; sem compilador nem Maven em produção
Dependências antes do código + cache mount	Mudar só `src/` não baixa dependências de novo
Jar em camadas	Um commit novo muda só a camada da aplicação (< 1 MB)
Usuário sem privilégio	Uma falha na aplicação não vira root no container
Healthcheck	`depends_on`, `up --wait` e o pipeline sabem quando a API está pronta de verdade
`exec java` + `MaxRAMPercentage`	JVM como PID 1 (shutdown gracioso no `docker stop`) e respeitando o limite de memória
`.dockerignore`	`.env`, `.git` e o app Android fora do contexto: segredo nunca vira camada

Imagem criada

Imagem publicada no GitHub Container Registry: `ghcr.io/alexcajefelix/fiap-aguia-branca`, digest `sha256:342ddef68af0…d6eb295ad8`, construída a partir do commit `1bde49c`. As cinco tags apontam para o **mesmo digest**: `sha-1bde49c` `main` `staging` `production` `latest` — as três últimas foram adicionadas pelos jobs de deploy depois de cada smoke test passar. Base `eclipse-temurin:21-jre-alpine`; 364 MB no total, com 40 MB de dependências e 688 kB de código da aplicação em camadas separadas.

Comandos usados

```
# build da imagem (local; no pipeline é o docker/build-push-action)
docker build -t aguiabranca-api:local --build-arg APP_VERSION=sha-local .

# ambiente local completo (dev): mongo + API, com seed
cp .env.example .env && docker compose up -d --build
docker compose ps && docker compose logs -f app

# staging e produção (os mesmos comandos que o pipeline executa)
JWT_SECRET=$(openssl rand -base64 48) scripts/deploy.sh staging    ghcr.io/alexcajefelix/fiap-aguia-
branca:sha-xxxxxxx
JWT_SECRET=$(openssl rand -base64 48) scripts/deploy.sh production ghcr.io/alexcajefelix/fiap-aguia-
branca:sha-xxxxxxx
#   = docker compose -p aguiabranca-<amb> -f docker-compose.yml -f docker-compose.<amb>.yml \
#       up -d --no-build --pull missing --remove-orphans --wait --wait-timeout 240
scripts/smoke-test.sh http://localhost:8082 staging    sha-xxxxxxx
scripts/smoke-test.sh http://localhost:8083 production sha-xxxxxxx

# promoção da imagem (sem rebuild) e limpeza
docker buildx imagetools create -t ghcr.io/…/fiap-aguia-branca:production ghcr.io/…/fiap-aguia-
branca:sha-xxxxxxx
scripts/deploy.sh staging --down
```

4 Prints do pipeline rodando

Execução real do pipeline: run #3 na `main`, disparado pelo merge do PR #62 (commit `1bde49c`, 13/10/2026), concluído em **5 min 13 s** com os quatro jobs verdes. Antes do merge, o próprio PR rodou as etapas de CI. Os trechos de log são do run #3, baixados pela API do GitHub (a página pública pede login para mostrar logs).

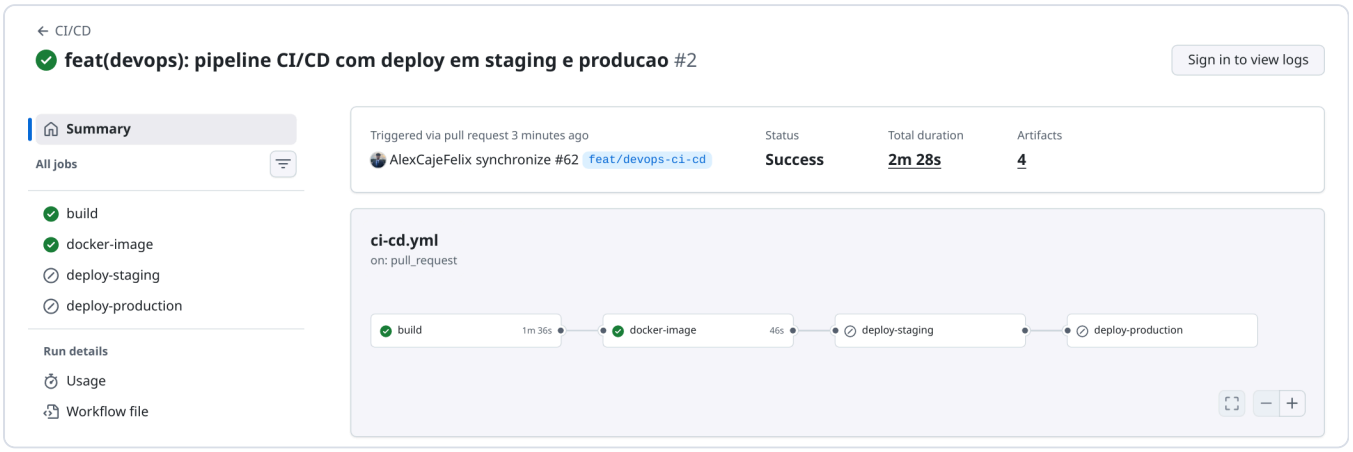


Figura P1. Pull request #62: `build` e `docker-image` verdes; os deploys são pulados em PR, como definido no workflow.

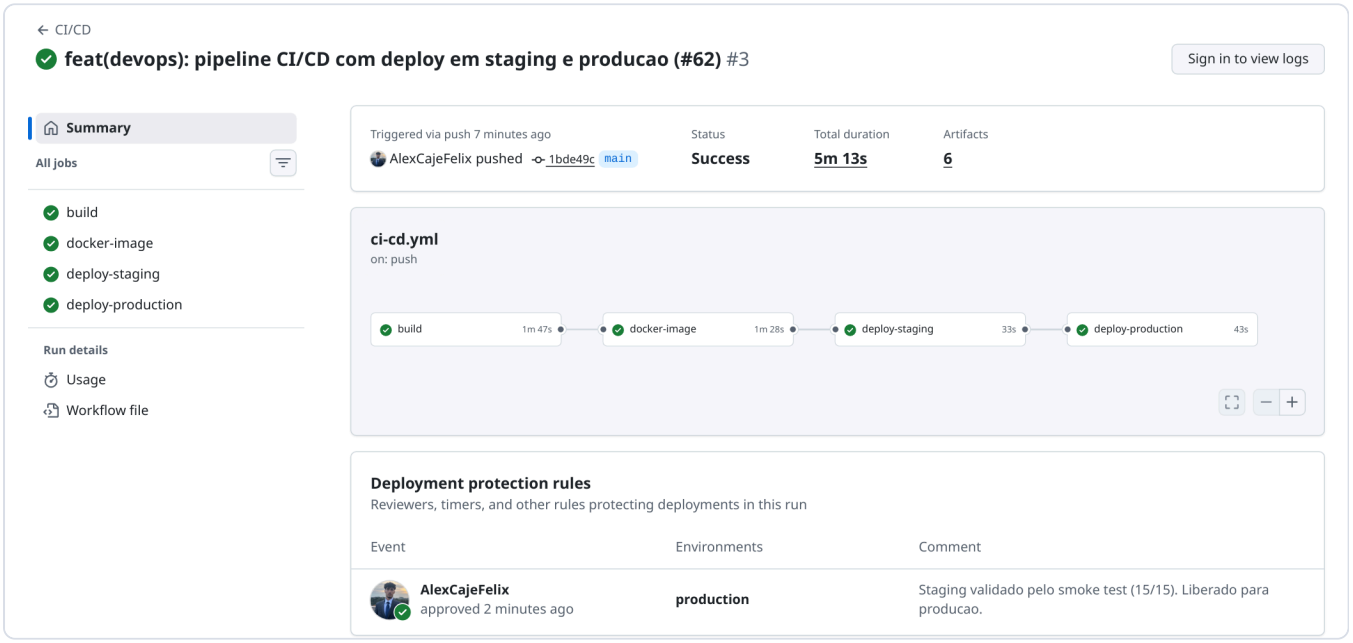


Figura P2. Run #3 na `main`: build → docker-image → deploy-staging → deploy-production, 5 min 13 s; a aprovação de produção fica registrada com o comentário do revisor.

GitHub Actions — run #3 (main, 1bde49c) — job build

```
▶ Run ./mvnw -B verify
[...]
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.418 s -- in domain.ai.AiConfigTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.985 s -- in domain.ai.IdeaSuggestionIntegrationTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.241 s -- in domain.auth.AuthFlowIntegrationTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.962 s -- in domain.auth.RefreshTokenIntegrationTest
[INFO] Tests run: 77, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 14.82 s -- in domain.auth.AuthorizationMatrixIntegrationTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.035 s -- in domain.auth.CorsDisabledIntegrationTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.424 s -- in domain.auth.ActuatorIntegrationTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.039 s -- in domain.auth.JwtSecretValidationTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.108 s -- in domain.auth.CorsEnabledIntegrationTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.824 s -- in domain.auth.RateLimitIntegrationTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.479 s -- in domain.project.DashboardQueryPlanTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s -- in domain.project.ProjectTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.222 s -- in domain.project.ProjectRepositoryTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.147 s -- in domain.strategy.StrategyIntegrationTest
[...]
[INFO] Tests run: 185, Failures: 0, Errors: 0, Skipped: 0
[INFO] Building jar: /home/runner/work/FIAP-AGUIA-BRANCA/FIAP-AGUIA-BRANCA/target/fiap-aguia-branca-0.0.1-SNAPSHOT.jar
[INFO] BUILD SUCCESS
[INFO] Total time:  01:26 min
```

Figura P3. Job `build`: testes por classe, **185 testes, 0 falhas**, jar gerado e `BUILD SUCCESS`.

GitHub Actions — run #3 (main, 1bde49c) — job docker-image

```
▶ Run docker/build-push-action@v7  (push: true)
#10 [runner 1/7] FROM docker.io/library/eclipse-temurin:21-jre-alpine@sha256:51ab5e3302e7141ce665ca3ea85e8b5cd648eafbc3c0c90dd79d6537684e4555
#13 [runner 2/7] WORKDIR /app
#14 [runner 3/7] RUN addgroup -S appgroup && adduser -S appuser -G appgroup     && mkdir -p /app/logs && chown appuser:appgroup /app/logs
#18 [builder 5/8] RUN sed -i 's/\r$//' mvnw && chmod +x mvnw
#19 [builder 6/8] RUN --mount=type=cache,target=/root/.m2 ./mvnw -B -q dependency:go-offline
#21 [builder 8/8] RUN --mount=type=cache,target=/root/.m2 ./mvnw -B -q clean package -DskipTests     && cp target/*.jar app.jar     && java -Djarm
ode=layertools -jar app.jar extract --destination extracted
#22 [runner 4/7] COPY --from=builder --chown=appuser:appgroup /app/extracted/dependencies/ ./
#23 [runner 5/7] COPY --from=builder --chown=appuser:appgroup /app/extracted/spring-boot-loader/ ./
#24 [runner 6/7] COPY --from=builder --chown=appuser:appgroup /app/extracted/snapshot-dependencies/ ./
#25 [runner 7/7] COPY --from=builder --chown=appuser:appgroup /app/extracted/application/ ./
#26 pushing manifest for ghcr.io/alexcajefelix/fiap-aguia-branca:sha-1bde49c@sha256:342ddef68af052860b95f8707d3c6fe8e5806c61cc7f1e42d27120d6eb295a
d8 1.2s done
#26 pushing manifest for ghcr.io/alexcajefelix/fiap-aguia-branca:main@sha256:342ddef68af052860b95f8707d3c6fe8e5806c61cc7f1e42d27120d6eb295ad8 0.4s
done
```

Figura P4. Job `docker-image`: estágios `builder` e `runner` do Dockerfile e push no GHCR das tags `sha-1bde49c` e `main`.

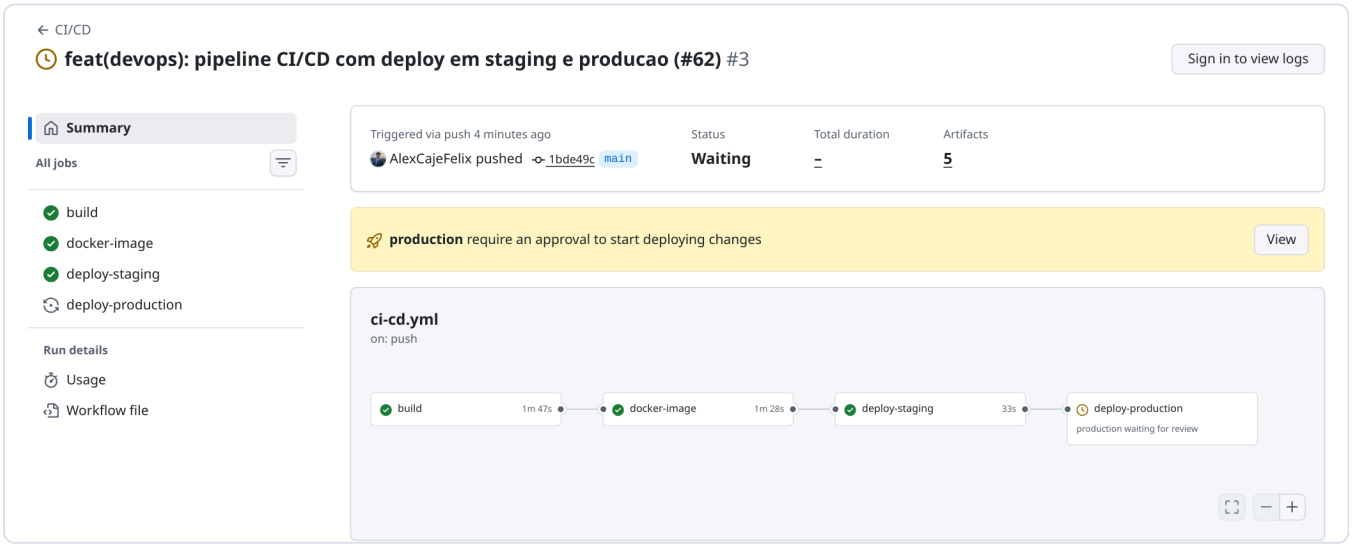


Figura P5. Portão de produção: com staging verde, o job `deploy-production` fica em *Waiting* até a aprovação do revisor do environment.

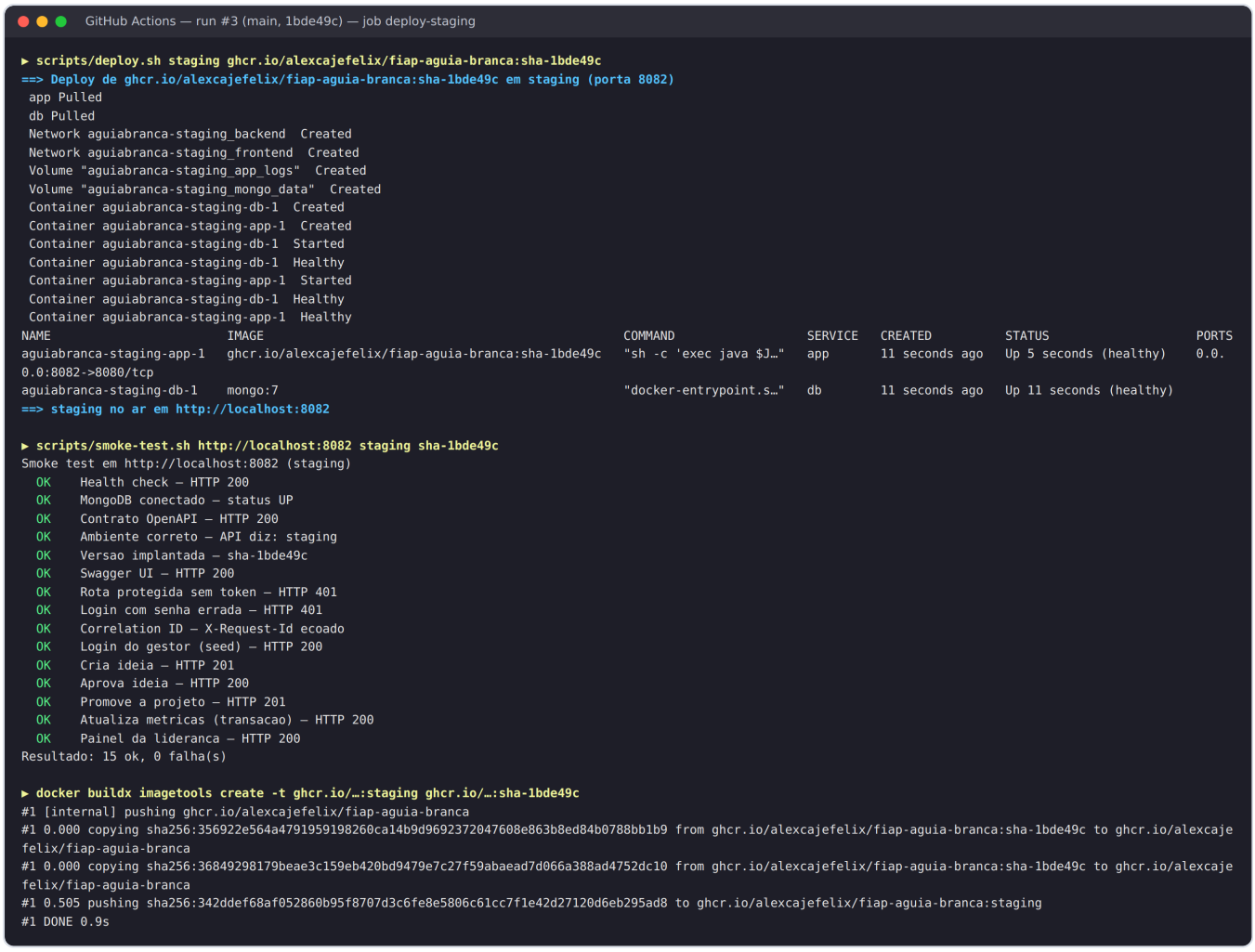


Figura P6. Job `deploy-staging`: o compose cria redes, volumes e containers, espera os healthchecks, o smoke test passa 15/15 e a imagem é promovida para `:staging` (mesmo digest).

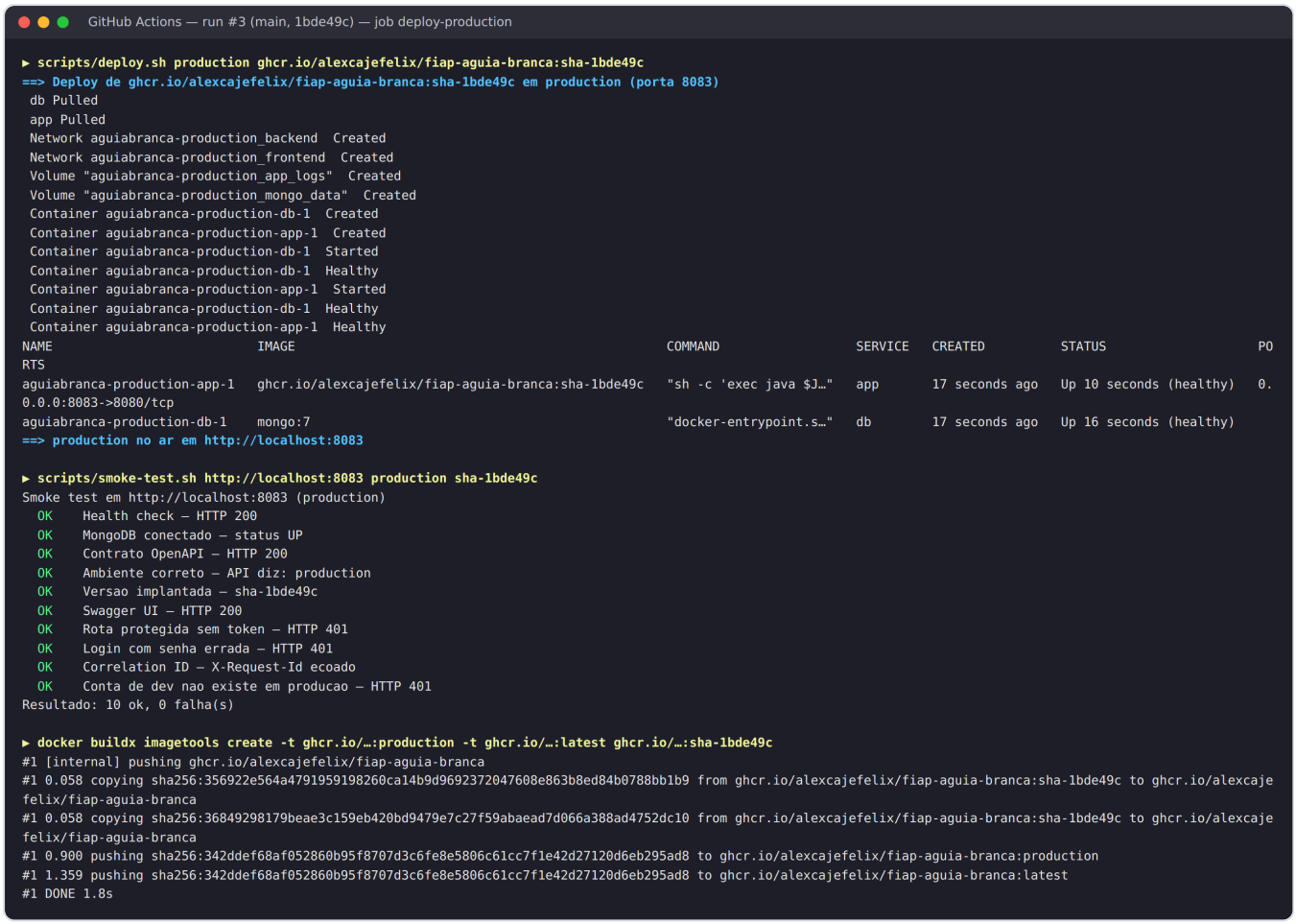


Figura P7. Job `deploy-production`: mesma imagem, override de produção, smoke test 10/10 (incluindo a recusa da conta de demonstração) e promoção para `:production` e `:latest`.

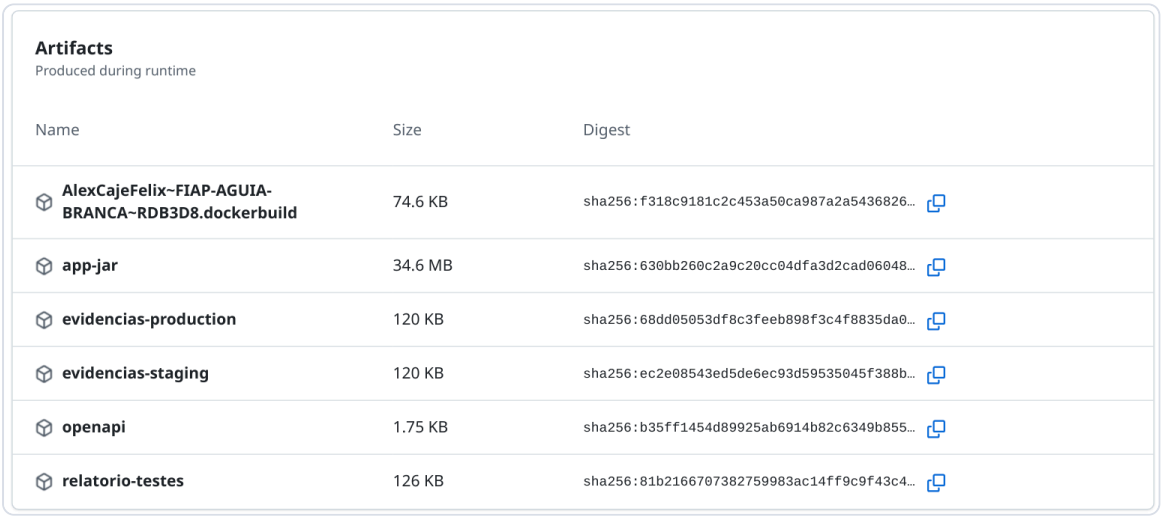


Figura P8. Artefatos publicados pelo run: jar, contrato OpenAPI, relatório de testes e as evidências de cada ambiente.

5 Staging e produção funcionando

As duas primeiras figuras foram geradas **pelo próprio pipeline**: depois do smoke test, o job abre o Swagger do ambiente recém-implantado no Chrome headless do runner e publica o print como artefato.

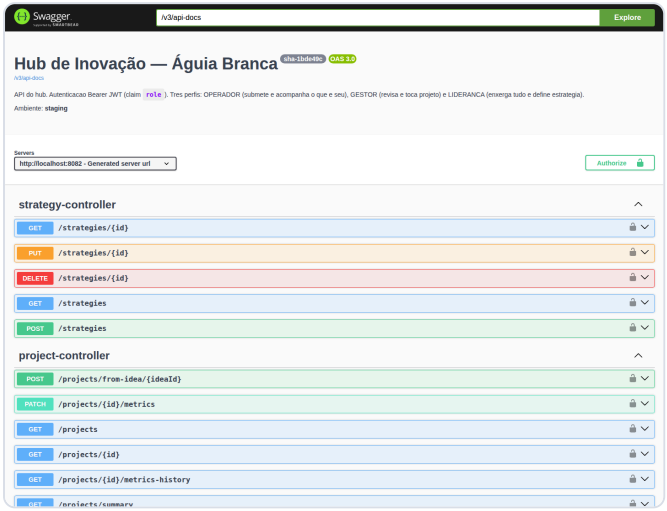


Figura E1. Staging implantado no run #3 (`:8082`): versão `sha-1bde49c`, *Ambiente: staging*.

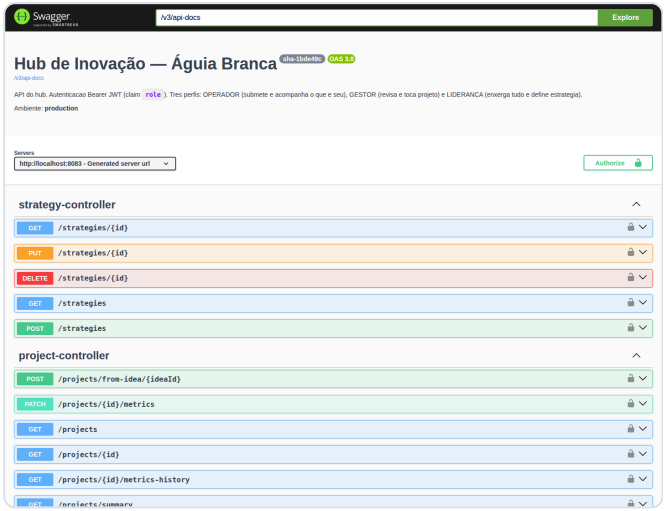


Figura E2. Produção implantada no run #3 (`:8083`): a mesma versão `sha-1bde49c`, *Ambiente: production*.

Respostas coletadas nos ambientes pelo pipeline

```
# evidencias-staging/staging-openapi-info.json
{ "info": { "version": "sha-1bde49c", ... }, "x-environment": "staging" }
# evidencias-production/production-openapi-info.json
{ "info": { "version": "sha-1bde49c", ... }, "x-environment": "production" }
# evidencias-production/production-health.json
{"status":"UP"}
```

```
terminal — historico de deploys dos GitHub Environments (API)

$ gh api repos/AlexCajeFelix/FIAP-AGUIA-BRANCA/deployments --jq '.[] | [.id, .environment, .ref, .sha[0:7], .created_at] | @tsv'
7051522427  production  main  1bde49c  2026-10-13T22:21:11Z
7051511417  staging     main  1bde49c  2026-10-13T22:20:35Z

$ gh api repos/AlexCajeFelix/FIAP-AGUIA-BRANCA/deployments/7051511417/statuses   # staging
2026-10-13T22:20:38Z  in_progress
2026-10-13T22:21:11Z  success

$ gh api repos/AlexCajeFelix/FIAP-AGUIA-BRANCA/deployments/7051522427/statuses   # production
2026-10-13T22:21:12Z  waiting
2026-10-13T22:21:41Z  queued
2026-10-13T22:21:41Z  queued
2026-10-13T22:21:44Z  in_progress
2026-10-13T22:22:27Z  success

$ gh api repos/AlexCajeFelix/FIAP-AGUIA-BRANCA/environments/production --jq '.protection_rules[] | .type'
required_reviewers  (revisor: AlexCajeFelix)
branch_policy
deployment_branch_policy: protected_branches=true
```

Figura E3. Histórico dos GitHub Environments: o deploy de staging vai de `in_progress` a `success`; o de produção passa por `waiting` (aprovação) antes de `success`. Produção exige revisor e só aceita branches protegidas.

Os dois ambientes lado a lado na máquina local

Com os mesmos scripts do pipeline, staging e produção foram implantados na máquina de desenvolvimento a partir da mesma imagem, cada um com containers, redes e volumes próprios. A rede `backend` é interna nos dois: um container nela não resolve nem alcança a internet.

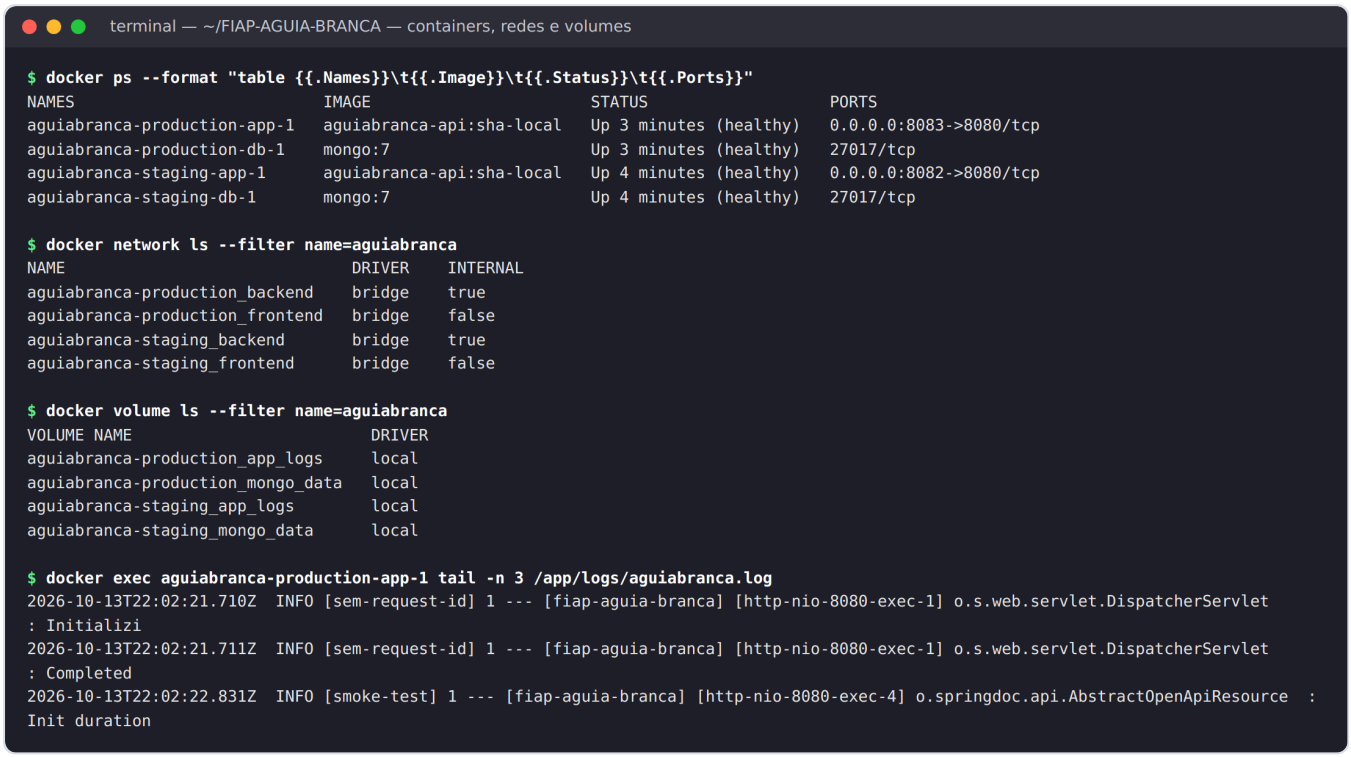


Figura L1. Containers saudáveis, redes (backend `internal=true`) e volumes de cada ambiente; o log em arquivo no volume `app_logs` mostra o `X-Request-Id` do smoke test.

```
terminal — ~/FIAP-AGUIA-BRANCA — smoke tests (staging e producao)

$ scripts/smoke-test.sh http://localhost:8082 staging sha-local
Smoke test em http://localhost:8082 (staging)
  OK    Health check — HTTP 200
  OK    MongoDB conectado — status UP
  OK    Contrato OpenAPI — HTTP 200
  OK    Ambiente correto — API diz: staging
  OK    Versao implantada — sha-local
  OK    Swagger UI — HTTP 200
  OK    Rota protegida sem token — HTTP 401
  OK    Login com senha errada — HTTP 401
  OK    Correlation ID — X-Request-Id ecoado
  OK    Login do gestor (seed) — HTTP 200
  OK    Cria ideia — HTTP 201
  OK    Aprova ideia — HTTP 200
  OK    Promove a projeto — HTTP 201
  OK    Atualiza metricas (transacao) — HTTP 200
  OK    Painel da lideranca — HTTP 200

Resultado: 15 ok, 0 falha(s)

$ scripts/smoke-test.sh http://localhost:8083 production sha-local
Smoke test em http://localhost:8083 (production)
  OK    Health check — HTTP 200
  OK    MongoDB conectado — status UP
  OK    Contrato OpenAPI — HTTP 200
  OK    Ambiente correto — API diz: production
  OK    Versao implantada — sha-local
  OK    Swagger UI — HTTP 200
  OK    Rota protegida sem token — HTTP 401
  OK    Login com senha errada — HTTP 401
  OK    Correlation ID — X-Request-Id ecoado
  OK    Conta de dev nao existe em producao — HTTP 401

Resultado: 10 ok, 0 falha(s)
```

Figura L2. Smoke tests: staging percorre o fluxo de negócio inteiro (15 verificações); produção prova que a conta de demonstração não existe (10 verificações).

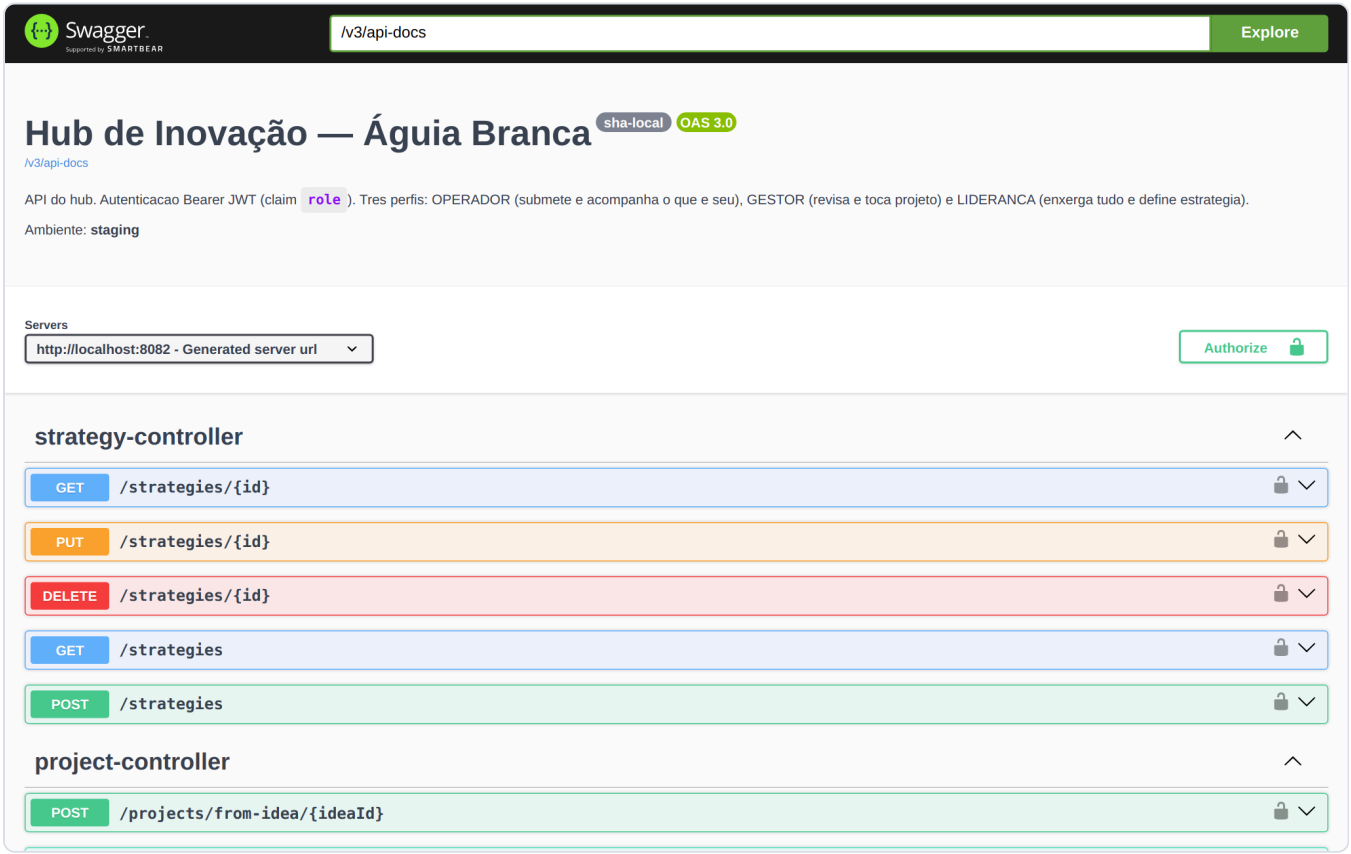


Figura L3. Swagger de staging (`localhost:8082`): o contrato informa a versão implantada e o ambiente.

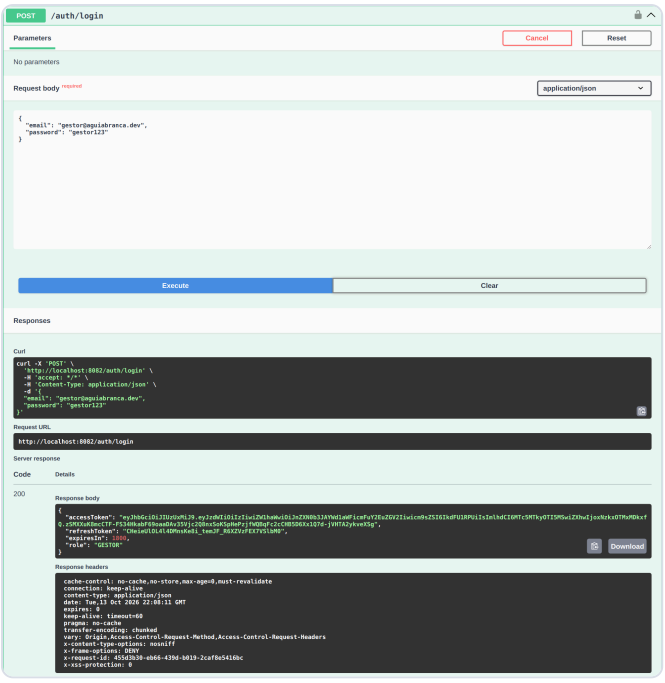


Figura L4. Staging: login do gestor pelo Swagger responde **200** com o par de tokens.

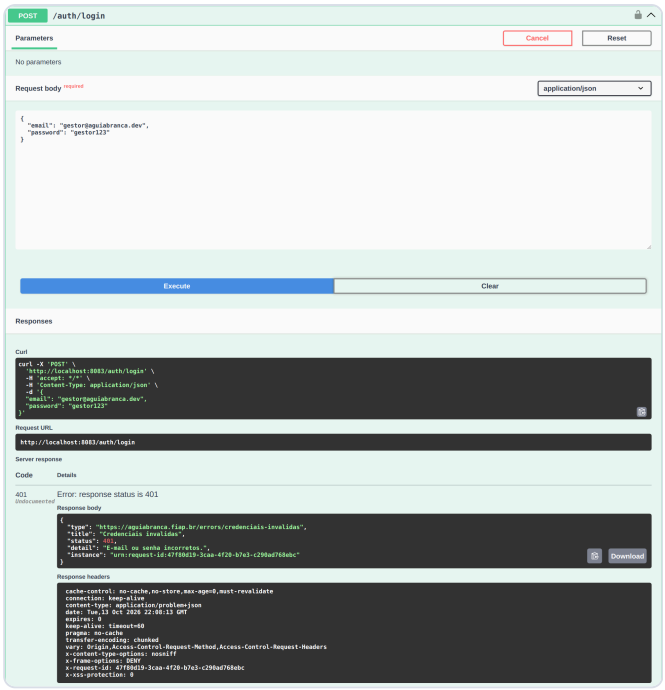


Figura L5. Produção: a mesma conta é recusada com **401** (RFC 7807): produção não tem seed.

```
terminal — ~/FIAP-AGUIA-BRANCA — ./mvnw verify

$ ./mvnw -B verify
[...]
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 12.11 s -- in shared.SwaggerUiAccessIntegrationTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.651 s -- in shared.RequestIdIntegrationTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.244 s -- in shared.OpenApiIntegrationTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.033 s -- in domain.auth.CorsDisabledIntegrationTes
t
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.038 s -- in domain.auth.JwtSecretValidationTest
[INFO] Tests run: 77, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 9.727 s -- in domain.auth.AuthorizationMatrixIntegr
ationTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.665 s -- in domain.auth.AuthFlowIntegrationTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.551 s -- in domain.auth.CorsEnabledIntegrationTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.557 s -- in domain.auth.RefreshTokenIntegrationTes
t
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.456 s -- in domain.auth.RateLimitIntegrationTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.273 s -- in domain.auth.ActuatorIntegrationTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.139 s -- in domain.project.ProjectRepositoryTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.778 s -- in domain.project.DashboardQueryPlanTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 s -- in domain.project.ProjectTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.744 s -- in domain.user.DevSeedIsolationTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.640 s -- in domain.user.DevSeedRunnerTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.603 s -- in domain.user.EnvironmentProfilesTest$Pr
oduction
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.641 s -- in domain.user.EnvironmentProfilesTest$St
aging
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.001 s -- in domain.idea.IdeaTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.702 s -- in domain.idea.IdeaIntegrationTest
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 2.008 s -- in domain.ai.IdeaSuggestionIntegrationTes
t
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.220 s -- in domain.ai.AiConfigTest
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.083 s -- in domain.ai.GeminiClientTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.011 s -- in domain.ai.SuggestionLimiterTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.530 s -- in domain.strategy.StrategyIntegrationTes
t
[...]
[INFO] Tests run: 185, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
[INFO] Total time:  43.451 s
```

Figura L6. `./mvnw -B verify` local: 185 testes, 0 falhas.

6 Desafios encontrados e soluções

Transação no MongoDB exige replica set

Problema: O `PATCH /projects/{id}/metrics` grava métrica e auditoria na mesma transação, que não existe num `mongod` avulso; num container novo o replica set ainda não está iniciado.

Solução: Mongo sobe com `--replSet rs0` e o próprio healthcheck executa `rs.initiate` na primeira subida; a API só inicia depois do banco *healthy*. O smoke test de staging exercita exatamente essa transação.

Staging precisa de contas; produção não pode ter nenhuma

Problema: O smoke test precisa logar para percorrer o fluxo, mas contas com senha conhecida em produção seriam uma brecha.

Solução: Profile `staging` carrega a massa de demonstração; o profile `prod` não. Um teste automatizado garante isso, e o smoke test de produção verifica a cada deploy que o login da conta de demonstração é recusado.

Saber se o deploy subiu a versão certa

Problema: Um healthcheck verde não diz qual imagem está rodando nem em qual ambiente.

Solução: A versão (tag do commit) vira variável de ambiente da imagem e aparece em `info.version` e `x-environment` do contrato OpenAPI; o smoke test compara com o que acabou de ser construído.

Override de porta no Compose sumia com a porta

Problema: No primeiro teste, `ports: !reset` seguido da nova porta deixou a API de staging sem porta publicada: `!reset` apaga o valor herdado e ignora o novo.

Solução: Trocado por `!override` (substitui a lista) e mantido `!reset []` só onde a intenção é remover — a porta do Mongo em staging e produção. Validado com `docker compose config`.

Dois ambientes no mesmo host

Problema: O compose original fixava `container_name`, o que impede subir staging e produção juntos.

Solução: Nome de container removido e cada ambiente com `-p aguiabranca-<ambiente>`: containers, redes e volumes ganham prefixo próprio e não colidem.

Testcontainers incompatível com Docker Engine recente

Problema: O Testcontainers trazido pelo Spring Boot 3.3.5 usava uma versão de API Docker recusada pelo Docker Engine 29, e os testes eram *pulados* em vez de falhar — build verde sem banco.

Solução: Testcontainers fixado em 1.21.4 no `pom.xml`; o pipeline roda os 185 testes contra MongoDB real.

Branch protection e nome do check

Problema: A `main` exige o status check chamado `build`; renomear o job no novo workflow travaria todo PR. Ela também exige uma aprovação, inclusive para administradores.

Solução: O primeiro job continua se chamando `build`. Para o merge desta entrega a exigência de aprovação para administradores foi desligada temporariamente e religada logo em seguida.

Detalhes do registry e do runtime

Problema: O GHCR recusa nomes com maiúsculas (o dono do repositório tem); a JVM, sem ajuste, dimensiona o heap pela RAM do host, não do container.

Solução: Nome da imagem em minúsculas com `${GITHUB_REPOSITORY,,}`; `-XX:MaxRAMPercentage=75` e limites de memória no compose.

Avisos de depreciação na primeira execução

Problema: O primeiro run do PR terminou verde, mas com avisos: `actions/checkout@v4` e `actions/setup-java@v4` rodam em Node.js 20, já descontinuado, e o rótulo `ubuntu-latest` migraria para Ubuntu 26 dias depois — mudando, sem aviso, o host onde os deploys rodam.

Solução: Actions atualizadas para as versões atuais (`checkout@v7`, `setup-java@v6`, `upload-artifact@v7`, `docker/*` v4/v7) depois de conferir as notas de versão, e runner fixado em `ubuntu-24.04`. O run seguinte saiu sem avisos de depreciação.

7 Checklist de entrega

Item	OK	Onde
Projeto compactado em .ZIP com estrutura organizada	✔	`FIAP-AGUIA-BRANCA.zip`
Dockerfile funcional	✔	`Dockerfile` — seção 3
docker-compose.yml ou arquivos Kubernetes	✔	`docker-compose.yml`, `docker-compose.staging.yml`, `docker-compose.prod.yml`
Pipeline com etapas de build, teste e deploy	✔	`.github/workflows/ci-cd.yml` — seções 2 e 4
README.md com instruções e prints	✔	`README.md` e `docs/evidencias/`
Documentação técnica com evidências (PDF ou PPT)	✔	este documento
Deploy realizado nos ambientes staging e produção	✔	seções 4 e 5 — run #3, environments `staging` e `production`

Estrutura entregue

```
FIAP-AGUIA-BRANCA/
├── Dockerfile                  imagem multi-stage da API
├── .dockerignore
├── docker-compose.yml          mongo + API (base e ambiente local)
├── docker-compose.staging.yml  override de staging
├── docker-compose.prod.yml     override de produção
├── .env.example                variáveis do ambiente local
├── deploy/                     staging.env.example, production.env.example
├── scripts/                    deploy.sh, smoke-test.sh
├── .github/workflows/ci-cd.yml pipeline CI/CD
├── src/                        código e testes (Java 21 / Spring Boot)
├── app-android/                app cliente (Kotlin + Compose)
├── docs/                       este PDF e as evidências (prints)
├── README.md
└── run.sh                      sobe o ambiente local com um comando
```

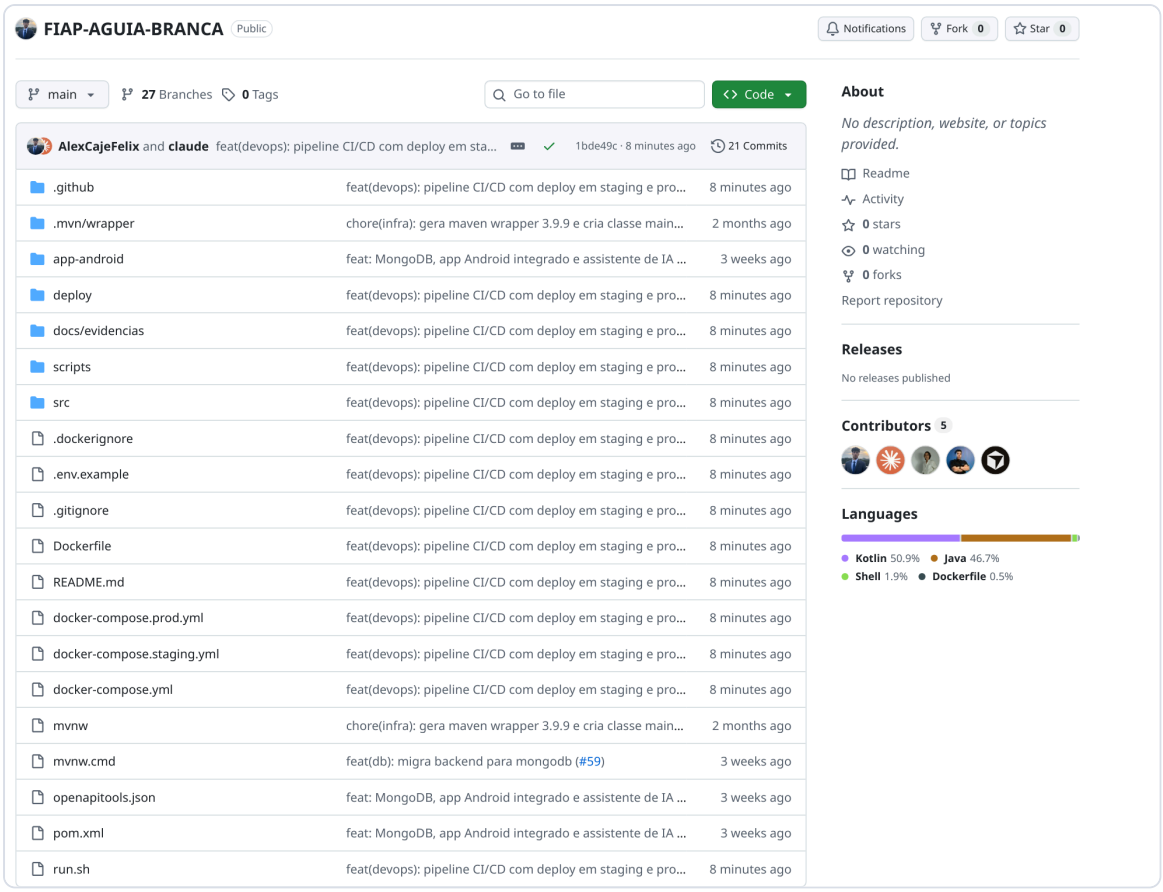


Figura C1. A mesma estrutura na `main` do repositório, com o commit do merge (`1bde49c`) verificado.